

TALLER # 2

Frontend y BD

Contenido

INTRODUCCIÓN A BOOTSTRAP	1
¿Qué es Bootstrap?	1
Relación entre Django y Bootstrap	2
Servidor de Bootstrap	2
URLs	3
Un solo archivo urls.py (en el proyecto)	3
Un archivo urls.py (por cada aplicación)	4
BASE DE DATOS	5
Sobre la base de datos	5
EJECUCIÓN DE ARCHIVOS	6
script de Python	6
Comando personalizado de Django	6
BIBLIOGRAFÍA	7

INTRODUCCIÓN A BOOTSTRAP

¿Qué es Bootstrap?

“Bootstrap is a powerful, feature-packed frontend toolkit. Build anything—from prototype to production—in minutes” (<https://getbootstrap.com/docs/5.3/getting-started/introduction/>). Bootstrap es un marco de trabajo (*framework*) de desarrollo *frontend* que se utiliza para crear **sitios web** (estático, informativo) y **aplicaciones web** (dinámico, acciones complejas de parte del usuario). Bootstrap proporciona una colección de herramientas y componentes de diseño basados en HTML, CSS y JavaScript que facilitan la creación de interfaces de usuario modernas y receptivas.

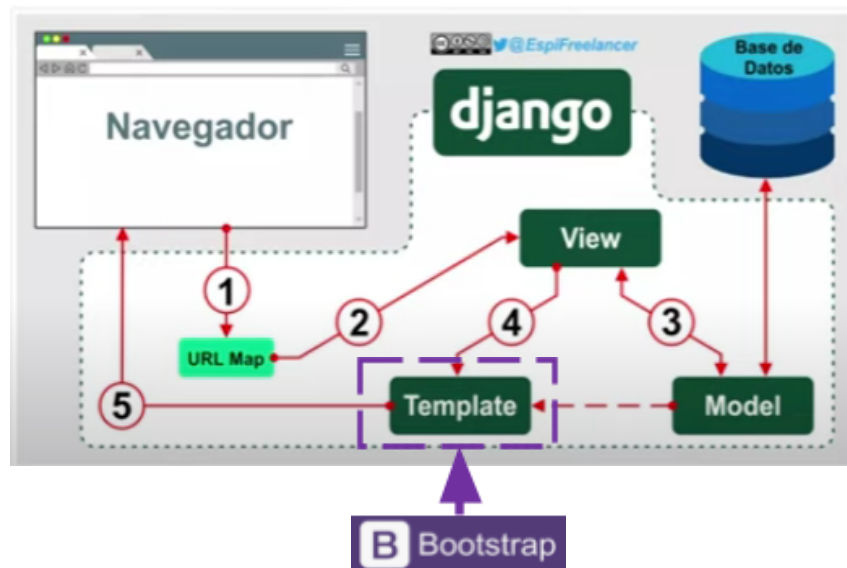
Algunas características Bootstrap son:

- Utiliza un sistema de rejilla (*grid system*) sensible que permite crear diseños flexibles y adaptativos para diferentes tamaños de pantalla y dispositivos.
- Proporciona una amplia gama de componentes preestilizados, como botones, formularios, barras de navegación, tarjetas, modales y más, que puede utilizar fácilmente en sus proyectos. Bootstrap incluye un conjunto de estilos prediseñados y utilidades de CSS que facilitan la personalización y la creación de diseños visualmente atractivos.

- Incluye varios *plugins* de JavaScript, como deslizadores de carrusel y acordeones, que pueden mejorar la funcionalidad del sitio web o aplicación web.

Relación entre Django y Bootstrap

Al combinar Django para la lógica del *backend* y Bootstrap para el diseño del *frontend*, es posible crear aplicaciones web modernas de manera eficiente, aprovechando la potencia de Django en el manejo de datos y la facilidad de uso de Bootstrap en la creación de interfaces de usuario atractivas y funcionales.



Servidor de Bootstrap

Incluiremos Bootstrap (<https://getbootstrap.com/>) vía CDN, en lugar de descargarlo e instalarlo localmente. Solo necesitas copiar un enlace <link> o <script> que apunta al archivo de Bootstrap almacenado en un servidor externo rápido y confiable.



Include via CDN

When you only need to include Bootstrap's compiled CSS or JS, you can use [jsDelivr](#). See it in action with our simple [quick start](#), or [browse the examples](#) to jumpstart your next project. You can also choose to include Popper and our JS [separately](#).

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.7/dist/css/b
```

```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.7/dist/js/b
```

CDN (*Content Delivery Network*) es una red de servidores distribuidos geográficamente que trabajan juntos para proporcionar contenido web de manera eficiente a los usuarios).

- *rel="stylesheet"* especifica que el recurso externo es una hoja de estilo.
- *integrity="..."* se utiliza para garantizar que el recurso no se haya modificado en tránsito. En este caso, se proporciona una cadena hash SHA-384.
- *crossorigin="anonymous"* especifica cómo el navegador debe manejar las solicitudes de recursos cruzados. En este caso, indica que las solicitudes de recursos externos deben realizarse sin enviar credenciales del usuario.

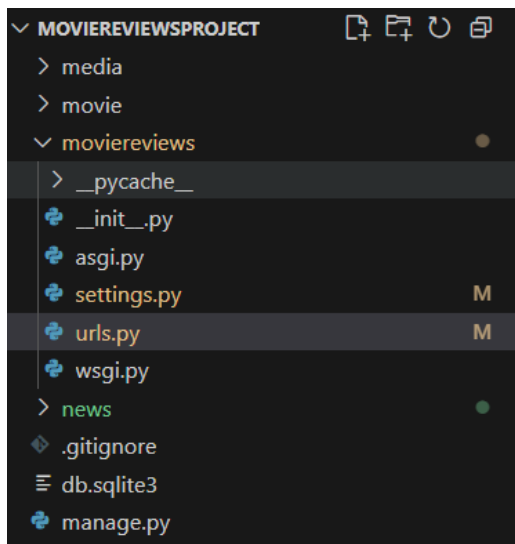
Algunas ventajas de usar CDN son:

- No tienes que descargar Bootstrap a tu computador.
- El navegador puede cargarlo más rápido porque los CDNs suelen estar optimizados y distribuidos en todo el mundo.
- Si el usuario ya visitó otra página que usa ese mismo CDN, es posible que ya tenga el archivo en caché, cargando aún más rápido.

URLs

Un solo archivo urls.py (en el proyecto)

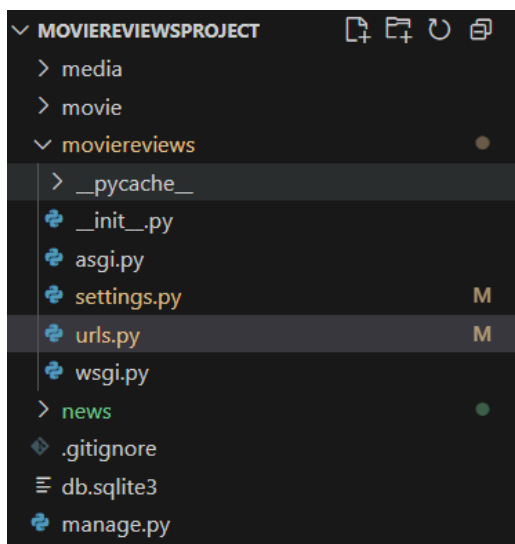
- Todas las rutas de todas las aplicaciones van en un único archivo.
- Es sencillo cuando el proyecto es pequeño y tiene pocas páginas.
- Cuando el proyecto crece con muchas apps, el archivo se vuelve largo, desordenado y difícil de mantener.



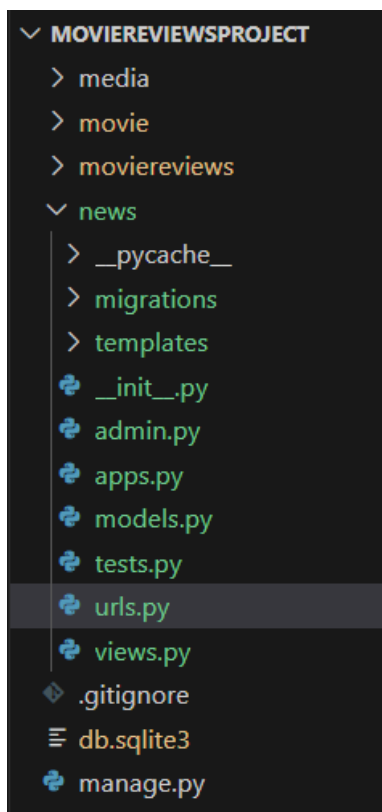
```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', movieViews.home),
    path('about/', movieViews.about),
]
```

Un archivo urls.py (por cada aplicación)

- Cada aplicación define sus propias rutas en su propio archivo *urls.py*.
- El archivo principal del proyecto (*urls.py* del proyecto) solo se encarga de incluir las rutas de cada app con *include()*.
- Mantiene el código organizado.
- Facilita la colaboración (cada desarrollador trabaja en el *urls.py* de su app).
- Si agregas nuevas apps, solo enlazas su archivo de rutas.



```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', movieViews.home, name='home'),
    path('about/', movieViews.about, name='about'),
    path('news/', include('news.urls')),
]
```



```
urlpatterns = [  
    path('', views.news, name='news'),  
]
```

BASE DE DATOS

Poblar automáticamente una base de datos tiene varias ventajas en el desarrollo de software. Por un lado, ahorra tiempo y esfuerzo al introducir datos de prueba o datos iniciales en la base de datos, lo que acelera el proceso de desarrollo y pruebas. Además, garantiza la consistencia y la integridad de los datos al eliminar errores humanos en la introducción manual de datos.

Sobre la base de datos

Actualmente, usamos una base de datos **SQL**, la cual se define en el archivo **settings.py**, en la zona de **DATABASES**. Por defecto, Django utiliza **SQLite** (<https://sqlite.org/>), que no es una base de datos propia de Django, sino un motor que viene incluido en la biblioteca estándar de Python. Esto permite que el framework la adopte como opción inicial porque es **ligera, sencilla y lista para usarse sin instalación adicional**.

SQLite resulta muy apropiada para proyectos pequeños, prototipos o pruebas rápidas, ya que se ejecuta desde un solo archivo (*db.sqlite3*) y basta con copiarlo para mover toda la base de datos. Sin embargo, **no está diseñada para aplicaciones con muchos usuarios concurrentes ni grandes volúmenes de datos**.

Límite	Valor estimado
Tamaño máximo de base de datos	≈ 281 TB

Límite	Valor estimado
Número teórico máximo de filas	$\approx 1.8 \times 10^{19}$ (2^{64})
Número práctico de filas	$\approx 2 \times 10^{13}$ (asumiendo páginas pequeñas)
Columnas por tabla (por defecto)	2,000
Columnas por tabla (max compilado)	32,767

En contraste, motores como **MySQL** son más **robustos y escalables**, adecuados para proyectos grandes. Ofrecen mayor rendimiento en consultas, manejo de múltiples conexiones simultáneas y características avanzadas como mayor seguridad, aunque requieren una configuración más compleja (instalar el servidor, crear usuarios, definir contraseñas, puertos, etc.).

Si en algún momento se necesita cambiar de base de datos, basta con modificar los atributos **ENGINE** y **NAME** (además de USER, PASSWORD, etc., según el motor).

```

83     DATABASES = {
84         'default': {
85             'ENGINE': 'django.db.backends.sqlite3',
86             'NAME': BASE_DIR / 'db.sqlite3',
87         }
88     }

```

EJECUCIÓN DE ARCHIVOS

script de Python

- Ejecuta un script de Python, fuera del entorno de Django.
- No necesita saber nada del proyecto Django, simplemente lee y escribe archivos como cualquier programa en Python.
- Se usa el comando: python seguido del nombre del archivo.py. Por ejemplo, **python cvs_to_json.py**

Comando personalizado de Django

- Ejecuta un comando personalizado de Django definido en la carpeta management/commands. En Django, los **comandos personalizados** se guardan dentro de una estructura específica para que Django los pueda reconocer y ejecutar.
- Se usa el comando: python manage seguido del nombre del comando (el cual normalmente coincide con el nombre del archivo). Por ejemplo, **python manage.py add_movies_db**
- Para ejecutar el comando, se debe ubicar en la carpeta que contiene el archivo *manage.py*.

BIBLIOGRAFÍA

Lim, G., Correa, D. Beginning Django 3 Development. Build Full Stack Python Web Applications. 2021.

<https://github.com/danielgara/bookdjango4.0>

Chacon, S., Straub, B. Pro Git. 2024. <https://git-scm.com/book/en/v2>

Octoverse. 2024. <https://octoverse.github.com/2022/top-programming-languages>

Moure, B. Curso de GIT y GITHUB desde CERO para PRINCIPIANTES.

<https://www.youtube.com/watch?v=3GymExBkKjE>

https://twitter.com/Harsa_Dash/status/1750741820135596036?s=20

<https://blog.hubspot.es/website/como-crear-footer-bootstrap>